

Search-Improved Level Generation :

以雙專家 MarioGPT 為先驗的 Gumbel AlphaZero 關卡規劃

Gumbel AlphaZero with a Structure/Action-Semantic-Path-Conditioned Level LLM as Policy Prior

Lung-Pin Chen

第一篇 目標：IEEE Transactions on Games (期刊)；會議備案 AIIDE / CoG

1 問題：自回歸關卡生成無法回頭

MarioGPT [1] 一類的關卡語言模型逐欄生成，每一步只看得到前綴。給定 STRUCT / PATH 條件時，模型無法預見「這段結構接上去之後，關卡是否仍可通關、未來的 PATH semantics 是否仍可實現」。當條件已不可能滿足時，只能硬生成或整張重抽。這是條件式序列生成的通病，在關卡生成中特別嚴重，因為可行性（可通關）與設計意圖的可執行性都不是單純的局部軟偏好。

本研究中的 **PATH 並非單純的地圖幾何曲線**。PATH 是沿著指定幾何路徑排列的一串 game-playing action tokens，例如 MOVE、JUMP、HIGH_JUMP、FALL、LAND 等，因此本身已具有明確的 traversal semantics。現有雙專家 MarioGPT 已能在同一生成過程中同時維持 STRUCTURE 合法性並 follow PATH condition；但兩者的融合發生在**兩個不同層級**：PATH 以 action tokens 在輸入序列層級注入（path token 直接取代地圖中的空白 tile 並以 forced decoding 固定其位置），STRUCT 條件則經 cross-attention 注入，**尚未顯式驗證「生成的 STRUCTURE 是否真的實現 PATH action semantics」**。例如 STRUCTURE 本身可以合法、PATH 也要求 HIGH_JUMP，但若跳躍區上方有 solid block，兩個條件各自看似合理，聯合起來卻不可執行。

因此本篇要補上的不是新的 PATH 表示，而是 **STRUCTURE-PATH semantic grounding**：把原本並列的 STRUCTURE / PATH conditioning，提升成可由遊戲動力學執行與驗證的 joint semantics，並讓搜尋前瞻目前的結構選擇是否會使後續 PATH action tokens 失去可實現性。

搜尋是自然的解法，但既有工作各有缺口：

- **MCMCTS** [2]：MCTS 引導 Markov chain 生成 Mario 關卡，reward 為可解性、缺口與敵人數。生成器弱、依賴隨機 rollout、搜尋結果不回饋生成器。
- **RL 生成器** [3, 4]：動作是固定的內容放置操作，動作空間不開放；Nam et al. 每步跑 agent 模擬計算 reward，成本高。
- **MarioGPT 的 novelty search**：進化式外圈，不是每步的前瞻。
- **LLM + MCTS** (RAP、LATS、TS-LLM [11])：集中在 reasoning / code，無 2D 空間結構、無可執行的 ground-truth 可行性 reward。

本研究的目標：把 AlphaZero 式的 expert iteration [6, 7]——搜尋當 expert、蒸餾回 policy——帶進條件式關卡 LLM，並處理 level generation 與 game-playing search 的兩個根本差異：**動作集合開放**（下一段結構有無限多種）、**目標是分布而非最優解**（需保留多樣性）。

定位。 與 Nam et al. [4]「quality-driven」(系統決定什麼是好關卡)不同，本研究是「intent-driven」：設計者以 STRUCT / PATH 指定意圖，其中 PATH 已編碼空間化的 action semantics；搜尋確保這些語義在生成的 STRUCTURE 中仍可被實現。與 novelty search 的差異是「design-meaningful diversity」(不同的可行設計)而非「surface-level variation」。本文將 PATH 視為明確的 *game-playing / traversal semantics*，但不主張這組 action tokens 已足以完整描述人類玩家的偏好、技巧、猶豫、探索或次佳決策；完整 player-behavior modeling 留待後續工作。

表 1: Comparison between standard MDP and our sampled-action MDP formulation.

Component	Standard MDP ($\mathcal{S}, \mathcal{A}, P, R$)	Sampled-Action MDP (Ours)
State space $\mathcal{S}$	$s_t \in \mathcal{S}$, typically a fixed representation	$s_t = (\ell_{1:t}, c)$: level prefix plus design intent, where PATH is a spatially grounded action-token sequence
Action space $\mathcal{A}$	Fixed, finite, state-independent set	<i>State-dependent</i> : $\mathcal{A}(s_t) = \{a_i\}_{i=1}^K$, sampled on the fly from $\mathcal{G}$ via SBS [10]; each action is a variable-length structural segment
Action prior	Not applicable; actions have no inherent probability	Each candidate a_i carries $\log \pi_\theta(a_i s_t)$, the LLM’s prior log-probability
Transition P	$P(s_{t+1} s_t, a_t)$, often stochastic	<i>Deterministic</i> : $s_{t+1} = \ell_{1:t} \oplus a_t$ (concatenation)
Reward R	$R(s_t, a_t, s_{t+1})$, typically per-step	<i>Terminal only</i> : feasibility requires both ordinary solvability and executable PATH semantics; soft terms measure STRUCT/PATH adherence
Policy	$\pi(a s)$ defined over fixed action set	$\pi_\theta(a s)$ defined by LLM; search constructs improved distribution π' over $\mathcal{A}(s_t)$
Decision rule	Typically $\arg \max$ (greedy) or PUCT	<i>Sampling from π'</i> , preserving diversity; not seeking a single optimal action
Objective	Find optimal policy π^* maximizing expected cumulative reward	Improve the <i>distribution</i> π_θ via expert iteration; diversity is a first-class objective

2 方法

2.1 Problem Formulation

We formulate conditional level generation as a sequential decision-making problem with a *sampled-action Markov decision process (MDP)*. Unlike conventional MDPs where the action set is fixed and state-independent, our formulation allows the action set to be dynamically proposed by a generative language model at each decision step, reflecting the open-ended nature of structural level design.

State Space. A state $s_t = (\ell_{1:t}, c)$ consists of:

- $\ell_{1:t} = (\ell_1, \ell_2, \dots, \ell_t)$: the prefix of the level generated so far, represented as a sequence of tokens or structural chunks;
- $c = (c_S, P)$: the design intent condition, consisting of a structural condition c_S (STRUCT) and a semantic path P (PATH) specified by the designer.

The structural condition c_S is encoded via cross-attention, while the path P is injected in-sequence as forced action tokens that replace blank (–) tiles at their designated map positions; both remain fixed throughout the generation process.

PATH Semantics. The PATH input is not merely a geometric polyline. We represent it as an ordered, spatially grounded action-semantic sequence

$$P = (p_1, p_2, \dots, p_m), \quad p_j = (r_j, u_j), \quad (1)$$

where r_j denotes a geometric region on the level and u_j is a game-playing action token such as MOVE, JUMP, HIGH_JUMP, FALL, or LAND. Because the LLM’s structural realization may shift geometry by a few tiles (column alignment, platform height), r_j is a *tolerant* region: a chunk-aligned box or a point with slack $\pm\delta$ tiles, so that a token counts as realized if the trajectory performs u_j anywhere inside r_j . The tolerance δ is a verifier hyperparameter whose sensitivity is reported in the experiments. The dual-expert model already conditions generation on this semantic sequence. The new problem addressed here is whether the generated structural realization actually *affords and executes* these action tokens in the specified order and regions.

Action Space and Proposal Mechanism. At each state s_t , the action set is not predefined but rather *sampled on the fly* from a dual-expert MarioGPT policy $\mathcal{G}$. Specifically, we draw K candidate next-segment actions via Stochastic Beam Search (SBS) [10], which performs Gumbel-top- k sampling without replacement at the sequence level:

$$\{(a_i, \log \pi_\theta(a_i \mid s_t))\}_{i=1}^K \leftarrow \text{SBS}(\mathcal{G}, s_t, K), \quad (2)$$

where each a_i is a variable-length token sequence representing a structural segment, and $\log \pi_\theta(a_i \mid s_t)$ is the standard autoregressive sequence log-probability assigned by the Structure Expert, summed over the non-forced (free) token positions of the segment; PATH tokens occupy their designated positions via forced decoding and do not contribute to $\log \pi_\theta$. The SBS procedure ensures that the K candidates are diverse and cover a range of structural realizations under the fixed path by performing Gumbel-top- k sampling without replacement at the sequence level (shifted Gumbel [10]), applied to the free-position logits of the Structure Expert.

Thus, the local action set at state s_t is defined as:

$$\mathcal{A}(s_t) := \{a_i\}_{i=1}^K, \quad (3)$$

which is a subset of the infinitely large space of all possible continuations. This sampled-action view is essential because enumerating all possible structural segments is intractable, yet we need a finite set for planning.

Transition Dynamics. The transition is deterministic. Upon selecting an action $a \in \mathcal{A}(s_t)$, the new state is obtained by concatenation:

$$s_{t+1} = (\ell_{1:t} \oplus a, c). \quad (4)$$

The process terminates when the level reaches a predefined target length or when an end-of-sequence token is appended.

Reward Structure. In contrast to typical MDPs that provide per-step rewards, our reward is *terminal only*. We distinguish ordinary level solvability from executable PATH semantics. Let

$$\text{Exec}_P(L, P) = \mathbb{K}[\exists \tau \in \mathcal{T}(L) : \tau \models P], \quad (5)$$

where $\mathcal{T}(L)$ is the set of player trajectories that obey the encoded Mario movement and collision rules, and $\tau \models P$ means that the trajectory realizes the ordered action tokens at their required geometric regions. Note that Exec_P already requires a start-to-goal trajectory, so it implies solvability; we keep $\text{Solvable}(L)$ as a separately reported quantity for comparison with playability-only baselines. Define the hard feasibility gate

$$F(L, P) = \text{Exec}_P(L, P). \quad (6)$$

The progress automaton (Sec. 2.2) also yields a graded quantity: the largest prefix of P that some physics-valid trajectory can realize,

$$\text{Prog}(L, P) = \frac{1}{m} \max\{j : \exists \tau \in \mathcal{T}(L) \text{ realizing } p_1, \dots, p_j\}, \quad \text{Exec}_P(L, P) = \mathbb{K}[\text{Prog}(L, P) = 1]. \quad (7)$$

We take $\text{Adh}_{\text{PATH}} := \text{Prog}(L, P)$: PATH adherence is thus the fraction of action tokens actually executed in order, a concrete quantity rather than an abstract agreement score. The terminal reward is *additive*, not gated:

$$r_T = F(L, P) + \lambda_s \cdot \text{Adh}_{\text{STRUCT}} + \lambda_p \cdot \text{Prog}(L, P), \quad (8)$$

where $\text{Adh}_{\text{STRUCT}}$ measures structural adherence. A multiplicative gate $F \cdot (\cdot)$ would assign identical zero reward to a level that misses one token and to one that ignores the intent entirely, leaving the value head and the completed-Q estimates without a gradient among infeasible candidates; the additive form keeps F as the hard criterion for *accepting* a level while preserving a signal for *learning*. Prog and Exec_P are not redundant with solvability: a level may remain globally solvable while violating the requested PATH semantics, for example when a requested HIGH_JUMP is blocked by an overhead tile but an alternative lower route still reaches the goal. No intermediate rewards are provided by the environment, making the problem sparse-reward; lookahead planning and the exact prefix check below compensate.

Policy and Improvement Objective. The generative model defines a prior policy $\pi_\theta(\cdot | s_t)$ over all possible continuations. However, since we only have access to the K sampled actions, we construct an *improved policy* π' over the local action set via search:

$$\pi'(a | s_t) \propto \exp(\log \pi_\theta(a | s_t) + \sigma(\hat{q}(a))), \quad (9)$$

where $\hat{q}(a)$ is an estimated completion value (see Sec. 2.4) and σ is a scaling function. Crucially, the final decision is made by *sampling* from π' rather than taking the argmax, preserving the distributional nature required for diverse level generation. This stands in contrast to traditional planning methods that seek a single optimal action.

Key Extensions over Standard MDP. Compared to the classical MDP tuple $(\mathcal{S}, \mathcal{A}, P, R)$, our formulation introduces three key extensions:

1. **State-dependent action sets:** $\mathcal{A}$ is replaced by $\mathcal{A}(s_t)$, which is sampled from the policy itself rather than being fixed a priori.
2. **Prior-aware improvement:** Each candidate action carries a prior log-probability $\log \pi_\theta(a | s_t)$, which is combined with search-derived values to form an improved distribution.
3. **Distributional objective:** The goal is not to find a single optimal action but to obtain an improved *distribution* π' that can be sampled from, ensuring expressiveness and diversity.

These extensions are necessary because level generation operates in an open action space where structural meaningfulness and designer intent must be balanced, and where diversity is a first-class objective rather than an afterthought.

Objective. Our overall objective is to iteratively improve the LLM policy π_θ by distilling the search-enhanced distribution π' back into the model:

$$\mathcal{L} = \mathbb{E}_{s \sim \rho} [\text{KL}(\pi'(\cdot | s) \| \pi_\theta(\cdot | s))] + \eta \cdot \text{KL}(\pi_\theta \| \pi_{\text{ref}}) + \mathcal{L}_{\text{value}}, \quad (10)$$

where ρ is the state distribution induced by the search policy, π_{ref} is the frozen original model acting as a regularizer, and $\mathcal{L}_{\text{value}}$ supervises the completion-value head. This expert-iteration loop is the core mechanism through which planning improves the generative model itself.

2.2 STRUCTURE–PATH Semantic Grounding and Executable Verification

The dual-expert LLM provides two complementary forms of design knowledge: STRUCTURE constrains what is placed in the map, while PATH specifies *where and how* the intended traversal should occur through spatially grounded action tokens. Their joint validity therefore cannot be reduced to two independent adherence scores. We introduce an executable compatibility relation

$$\text{Compat}(L, P) = \text{Exec}_P(L, P), \quad (11)$$

which asks whether the generated structure truly affords the intended PATH semantics.

The verifier is implemented as a physics-aware A^* search over player states rather than a tile-grid shortest path. A player state may include position, velocity, grounded status, and other variables required by the encoded Mario movement model. The PATH sequence is compiled into an ordered progress automaton; the search state is augmented with a progress index j , and the index advances only when the trajectory realizes action token u_j in region r_j . Thus the same search infrastructure supports two distinct queries:

$$\text{Solvable}(L) : \exists \text{ any physics-valid trajectory from start to goal,} \quad (12)$$

$$\text{Exec}_P(L, P) : \exists \text{ a physics-valid trajectory from start to goal that also satisfies } P. \quad (13)$$

For example, a level containing a legal ceiling block may still be structurally valid, but if that block intersects the required HIGH_JUMP arc then $\text{Exec}_P = 0$, even when another route keeps $\text{Solvable}(L) = 1$.

Exact prefix verification at internal nodes. The verifier is not confined to terminal levels. For a partial level $\ell_{1:t}$, every PATH token whose region r_j lies inside the generated span is already decidable: we run the same player-state A^* up to the current generation frontier and obtain

$$\text{Prog}_{\text{pre}}(\ell_{1:t}, P) = \frac{1}{m} \max \{j : r_j \subseteq \text{span}(\ell_{1:t}), \exists \tau \text{ realizing } p_1, \dots, p_j\}, \quad \text{Dead}(\ell_{1:t}, P) = \mathbb{K}[\exists j : r_j \subseteq \text{span}(\ell_{1:t}), \text{not } P] \quad (14)$$

$\text{Dead} = 1$ is an exact, rule-derived certificate that the prefix can no longer realize P ; such a candidate is removed from $\mathcal{A}(s_t)$ (hard prune) before any learned estimate is consulted. The learned completability value (Sec. 2.4) is therefore responsible only for the *future* tokens, those whose regions have not yet been generated. This exact-prefix / learned-future division makes the search auditable: every pruning decision inside the tree can be traced either to a verifier certificate or to a value estimate, and the two are reported separately. The prefix check is cheaper than the terminal one since the search stops at the generation frontier and results are memoized along the tree path.

This verifier provides a hard guarantee only with respect to the game rules and semantic predicates explicitly encoded in the player-state model; it is not a claim that PATH tokens fully model human behavior. In this paper, PATH semantics represent designer-specified traversal/game-playing features. Richer human behavior—skill, hesitation, exploration, risk preference, or suboptimal action choice—is outside the scope of the first paper.

2.3 Sampled-action MDP

狀態 $s_t = (\ell_{1:t}, c)$ ：已拼接的關卡前綴與條件圖。動作 a_t ：下一段結構（token 序列）。動力學確定： $\ell_{1:t+1} = \ell_{1:t} \oplus a_t$ 。動作集合由雙專家 MarioGPT $\mathcal{G}$ 在每個節點抽 K 個候選構成：

$$\{(a_i, \log \pi_\theta(a_i | s_t))\}_{i=1}^K \leftarrow \text{SBS}(\mathcal{G}, s_t, K), \quad (15)$$

其中 SBS 為 Stochastic Beam Search [10]——序列層級的 Gumbel-Top- k 不放回抽樣，保留標準自回歸 log-prob（僅累積未被 forced 的自由位置；path token 以 forced decoding 固定，不計入 $\log \pi_\theta$ ）。這是把 Sampled MuZero [8] 的「以 prior 樣本定義動作集合」用在生成模型上。序列層級的 shifted Gumbel 噪音使 K 個候選在固定 path 下涵蓋不同的 structural realization，提供可解釋的多樣性軸。

2.4 Gumbel 搜尋與 Semantic Completability

每節點以 Sequential Halving 在 K 個候選上分配模擬預算 n (Gumbel MuZero [9])。葉節點以 *semantic completability value* 評估：

$$v_\phi(s_t) = \Pr \left[\exists L_{1:T} : L_{1:T} \text{ structurally valid} \wedge \text{Solvable}(L_{1:T}) \wedge \text{Exec}_P(L_{1:T}, P) = 1 \right]. \quad (16)$$

v_ϕ 接在凍結 $\mathcal{G}$ backbone 最後一層 hidden state 上（共用 cross-attention 條件記憶）。它預測的不是「目前片段看起來好不好」，而是從目前 **partial level** 出發，是否仍存在能讓 **STRUCTURE** 與未來

PATH action semantics 聯合成立的完成。由於 Eq. (14) 已精確判定前綴內的 token， v_ϕ 只需估計尚未生成區域的 token 可實現性；輸入另附 Prog_{pre} 與尚待實現的 token 序列 $p_{j+1..m}$ 。終端 reward 使用 Eq. (8)，由 physics-aware A^* 提供 Prog 與 Exec_P 的 ground truth。

存在量詞的 value target。 v_ϕ 是存在性命題，AlphaZero 慣用的「該狀態實際完成的結果」作為 target 有單邊雜訊：一次失敗的完成不證明可行完成不存在。因此 target 定義為該前綴在本輪搜尋中所有已探索完成的最大值——任一完成達到 $\text{Exec}_P = 1$ 即標為 1（max-backup 而非 mean-backup）；負標籤（無任何已探索完成可行）以較低權重納入，並隨搜尋預算增加而上調，因為預算越大負標籤越可信。Dead = 1 的前綴則為確定的負樣本，權重為 1。

每個候選 $\ell \oplus a$ 先經 Eq. (14) 的 prefix verifier：Dead = 1 者直接自 $\mathcal{A}(s_t)$ 移除。改良分布以 completed Q-values 計算： $\pi'(a) \propto \exp(\log \pi_\theta(a) + \sigma(\hat{q}(a)))$ ，未展開的候選以 v_ϕ 補值。下一段從 π' 抽樣而非取 argmax，以保留分布性。搜尋的核心功能因此不只是提高通關率，而是**前瞻目前的 structural decision 是否會破壞未來 PATH semantics 的可實現性**。局部上合法的片段仍可能在數個 chunk 後造成必要 landing region、jump clearance 或 action ordering 無法滿足，這正是自回歸生成缺乏回頭能力時最難處理的錯誤。

為什麼是 Gumbel。每次展開是一次 LLM 呼叫，模擬預算註定小 ($n \in [8, 32]$)。Gumbel MuZero 的核心保證正是低預算下 π' 在期望上不劣於 π_θ 。PUCT 在此預算下無此保證。此 policy-improvement 性質是相對於已抽樣的候選集合與 value/Q 估計假設，並不同於對所有可能 continuation 的全域最優保證；輸出關卡的 hard semantic guarantee 來自終端 executable verifier。

2.5 Completed-Q 蒸餾：search-improved LLM

搜尋後的 π' 蒸餾回 $\mathcal{G}$ ：

$$\mathcal{L} = \mathbb{E}_s [\text{KL}(\pi'(\cdot | s) \| \pi_\theta(\cdot | s))] + \eta \text{KL}(\pi_\theta \| \pi_{\text{ref}}) + \mathcal{L}_{\text{value}}. \quad (17)$$

雙專家 backbone 凍結，僅在 Structure Expert（即提案結構段的 policy）上更新附加的 LoRA adapter；Path Expert 全程凍結。 π_{ref} 為凍結的原模型，KL 項防止蒸餾後分布塌縮與條件遵從退化。由於搜尋 target 已納入 STRUCTURE-PATH executable consistency，蒸餾的目標不只提高一般 playability，也要降低「STRUCT 合法但 PATH action semantics 無法執行」的 incompatibility。主張：蒸餾後的 $\mathcal{G}$ 在不搜尋時仍優於原模型、semantic executability 提升且多樣性未崩——這是 AlphaZero 式結果在 PCG 上的首次展示。

2.6 理論命題（規劃中）

Proposition 1 (非正式). 設動作集合由 SBS 自 π_θ 不放回抽出 K 條序列，並以 *Sequential Halving* 與 *completed-Q* 建構 π' 。則在 *Gumbel MuZero* 的假設下（ $\hat{q}$ 為無偏估計）， $\mathbb{E}_{a \sim \pi'}[q(a)] \geq \mathbb{E}_{a \sim \pi_\theta}[q(a)]$ 對序列層級動作仍成立。

Gumbel MuZero 的證明針對有限離散動作集合；SBS 提供的正是「從 π_θ 的 Gumbel-Top- k 不放回樣本」，故可沿用其論證。此處候選的 $\log \pi_\theta$ 為 Structure Expert 的標準自回歸序列 log-prob（僅在自由位置累積；path-token 由 forced decoding 固定，不計入）。此處的 q 以 semantic-feasibility-aware return 定義。若證出，是獨立於 Mario 的方法貢獻；期刊版本可含此附錄，若未完成則以實驗支持。需明確區分：此命題是 sampled candidate set 上的 policy-improvement 命題，而「接受的關卡符合已編碼 PATH semantics」則由 executable verifier 保證。

2.7 方法圖

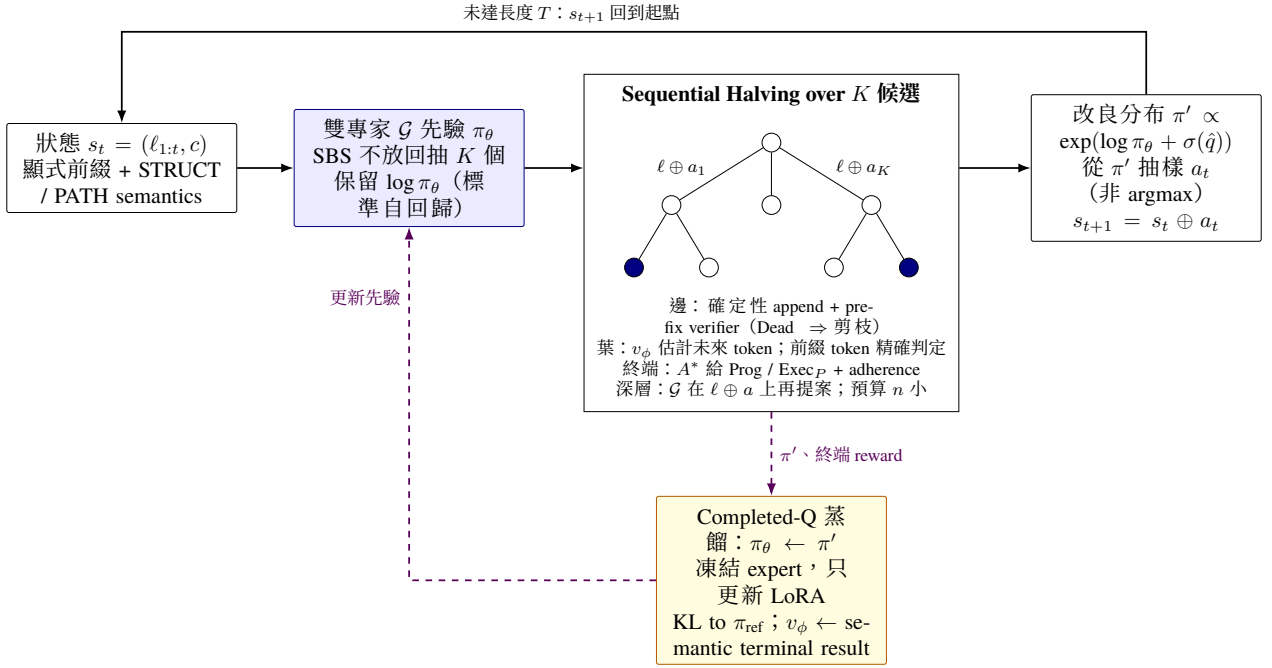


圖 1: 每一步由雙專家 MarioGPT 以 SBS 提案 K 個候選，Sequential Halving 以 semantic completability value 評估葉節點；終端由 physics-aware A^* 分別驗證一般通關與 PATH action semantics 的可執行性，再結合 STRUCT/PATH adherence。從 completed-Q 改良分布抽樣下一段。虛線為 AlphaZero 式外圈： π' 蒸餾回 Structure Expert 的 LoRA。

3 貢獻

1. **STRUCTURE-PATH semantic grounding**：PATH 已是「幾何位置 + action tokens」的 traversal semantics；本篇進一步把它與生成 STRUCTURE 建立 executable relation，區分「STRUCT 合法」「PATH adherence」與「STRUCTURE 是否真的實現 PATH semantics」。
2. **開放動作集合的 Gumbel 搜尋**：SBS (序列層級 Gumbel-Top- k) $\times$ Sequential Halving $\times$ completed-Q，在動態 LLM proposal action set 上前瞻 structural decisions 對未來 PATH semantics 可實現性的影響。
3. **Exact-prefix / learned-future completability**：前綴內的 PATH token 由 player-state A^* 精確判定 (Dead 證書即硬剪枝)，尚未生成區域的 token 由接在凍結 backbone 上的 semantic completability value 估計；存在性 value 採 max-backup target。取代昂貴 rollout，並使每個剪枝決定可追溯到證書或估計。
4. **Search-improved level LLM**：completed-Q 蒸餾回附加於 Structure Expert 的 LoRA adapter，使 executable structure-path consistency 由 search-time improvement 進一步內化到生成模型；蒸餾後模型不搜尋仍應優於原模型且多樣性未崩。
5. **Distribution, not argmax**：從 π' 抽樣並以 expressive range 驗證，把「目標是改善可行設計的分佈」納入方法，而非只產生單一最佳關卡。 A^* 衍生的 branching、structural difficulty、optimal-play complexity 則保留為 behavioral evaluation，而非主要演算法貢獻。

4 實作：雙專家 MarioGPT + LightZero

LightZero [12] 無 Gumbel AlphaZero，故取其 Gumbel MuZero 的搜尋 (Gumbel-Top- k 、Sequential Halving、completed-Q、reanalyse)，把 g 換成真實 append + 重新編碼。兩個衝突與解法：(1) 固定動作空間 $\rightarrow$ 節點局部動作表 (索引 $0..K-1$ 對應該節點的 K 個候選，SBS 的 $\log \pi_\theta$ 作為 policy logits)；(2) 批次張量推論 vs 自回歸解碼 $\rightarrow$ 第一階段 Python 樹，SBS 改多葉節點批次解碼。

- S1. 凍結雙專家 $\mathcal{G}$ ，實作 $\text{propose}(s, K)$ ：SBS 以序列層級 shifted Gumbel 不放回抽樣， $\log \pi_\theta$ 為 Structure Expert 的標準自回歸序列 log-prob（僅累積自由位置；path token 為 forced，不計入）；單元測試候選不重複、log-prob 與 teacher-forcing 一致。
- S2. 明確化 PATH semantic interface：將每個 PATH token 表示為 (r_j, u_j) ，其中 r_j 為幾何 region、 u_j 為 action token；建立 token-to-predicate 對應與 ordered progress automaton。
- S3. 實作 physics-aware A^* verifier：player state 含 movement / collision 所需變數，搜尋節點擴充 PATH progress index；輸出 $\text{Solvable}(L)$ 、 $\text{Prog}(L, P)$ 、 $\text{Exec}_P(L, P)$ ，並支援停在生成邊界的 prefix 模式輸出 Prog_{pre} 與 Dead（沿樹路徑 memoize）。region 容差 δ 為參數。建立 semantic unit tests，例如 HIGH_JUMP 上方被 block 阻擋、gap 超過可跳距離、landing 被占據、required FALL/JUMP 次序不可能、以及「前綴 Dead 但整體仍 Solvable」的案例。
- S4. LevelGenEnv：observation = (ℓ, c) ，step(i) append 局部表第 i 個切片，達目標長度或結束符終止；終端 reward 使用 Eq. (8)（加法， $\text{Adh}_{\text{PATH}} = \text{Prog}$ ）；step 對候選先呼叫 prefix verifier，Dead 者自局部動作表移除。
- S5. Value head v_ϕ 接在 backbone 上，輸入含 Prog_{pre} 與待實現 token；以 max-backup 的存在性 target 監督（Dead 前綴為確定負樣本），初期使用短關卡（ $T \leq 8$ ）curriculum。
- S6. 小規模跑通： $K = 4$ 、 $n = 8$ 、Python 樹；確認 completed-Q 能避開典型 STRUCTURE-PATH incompatibility，並驗證蒸餾在 LoRA 上有效且 adherence / diversity 不退化。
- S7. 迴圈：搜尋生成 $\rightarrow A^*$ solvability + PATH executability + adherence 結算 $\rightarrow$ replay $\rightarrow$ 更新 v_ϕ 與 Structure Expert 的 LoRA（KL to reference） $\rightarrow$ reanalyse；蒸餾 0/1/3/5 輪。
- S8. 擴大： $K \in \{4, 8, 16\}$ 、 $n \in \{8, 16, 32\}$ 、 T 全長；效能瓶頸確認後決定是否改 C++ 樹。

5 實驗設計（期刊等級）

表 2: 實驗設計

項目	設定	回答的問題
基線	(a) $\mathcal{G}$ 純採樣；(b) $\mathcal{G}$ + rejection sampling（同 LLM 呼叫數）；(c) MCTS + 隨機 rollout（MCMCTS 風格，無 v_ϕ ）；(d) $\mathcal{G}$ + novelty search（MarioGPT 原版）；(e) PUCT + i.i.d. 抽樣（非 Gumbel）；(f) PCGRL 風格 RL 生成器（固定動作空間）	搜尋增益是否超過同算力重抽；learned value 是否勝過 rollout；Gumbel 是否勝過 PUCT；開放動作空間是否勝過固定
指標	playability (A^* 通關率)；PATH semantic executability (Exec_P)；STRUCTURE-PATH incompatibility rate；STRUCT / PATH adherence；per-token semantic satisfaction；expressive range 與 pairwise diversity；每張可用關卡的 LLM 呼叫數與 wall-clock；search-derived behavioral 指標（branching、structural difficulty、optimal-play complexity）	品質、可控性、joint semantic consistency、多樣性、成本、設計意義
主結果 1	有搜尋 vs 無搜尋，各基線同算力；同報 solvability 與 Exec_P	搜尋是否不只提高通關率，也提高 STRUCTURE 對 PATH action semantics 的未來可實現性
主結果 2	蒸餾 0/1/3/5 輪後的 $\mathcal{G}$ ，「無搜尋」評估；同時報告 Exec_P 、incompatibility、adherence 與 expressive range	search-discovered semantic consistency 是否被內化到 LLM，且模型未變保守
消融	K ；預算 n ；有無 v_ϕ ；有無 prefix verifier 剪枝（純 v_ϕ vs exact-prefix + v_ϕ ）；乘法 gate vs 加法 reward；max-backup vs mean-backup value target；solvability-only verifier vs +PATH semantic verifier；semantic PATH vs geometry-only PATH；completed-Q vs visit-count 蒸餾； π' 抽樣 vs argmax；序列層級 Gumbel-Top- k 不放回 vs i.i.d. 抽樣（多樣性來源）；動作粒度 column / chunk / semantic；蒸餾範圍（全參數 / LoRA）；KL 係數 η	驗證 semantic grounding 是否必要；各元件必要性；低預算下 Gumbel 的優勢來源；粒度取捨；條件遵從保持
可解釋性	搜尋樹分支依結構/路徑主導性標色；追蹤 PATH token progress；分析「STRUCT 合法但 PATH semantics 不可執行」的回頭點，例如 blocked high jump / impossible landing；每個剪枝決定標記為「verifier 證書」或「 v_ϕ 估計」	搜尋在哪裡修復 STRUCTURE-PATH semantic incompatibility；修復有多少來自精確證書、多少來自學習估計
一般性	換條件類型（僅 STRUCT / 僅 semantic PATH / STRUCT+PATH；部分 PATH token 指定、其餘設為 wildcard），並比較不同 action-token vocabulary 的可驗證性	intent-driven semantic framework 的延展性，而不把未指定維度改成系統自動決定的 quality objective
Semantic stress tests	系統化注入 blocked high jump、insufficient jump distance、blocked landing、錯誤 FALL/JUMP ordering、局部可行但後續 token 不可完成等案例；region 容差 $\delta \in \{0, 1, 2, 4\}$ tiles 的敏感度	verifier 是否能抓 local executable error；prefix 剪枝抓到多少比例的 delayed failure、剩餘由 Gumbel lookahead 處理多少； δ 對 Exec_P 與多樣性的影響
人類評估	10–20 位受試者對「蒸餾前 / 蒸餾後 / 搜尋」關卡的可玩性、指定 traversal/action intent 辨識與自然度評分	executable PATH semantics 的客觀指標是否與玩家感知一致；不宣稱完整 human-behavior model
統計	每設定 5 seeds；報告均值與 95% CI；配對檢定	期刊審稿要求
風險	value 初期訊號稀疏 → 短關卡 curriculum；蒸餾使 LLM 保守 → KL to reference + 同報 diversity；增益不超過 rejection sampling → 提高 T 與條件難度以凸顯回頭價值 0	—

6 與第二篇的關係

本篇動力學已知、reward 僅終端，是第二篇（Gumbel Stochastic Direct MuZero：玩家為環境、直接預測玩家行為、chance node、epistemic exploration）的必要前置：搜尋骨架、先驗、蒸餾、semantic completeness 與 executable PATH verification 全部由本篇提供並經驗證。第一篇的 PATH semantics 是設計者指定的 **traversal/action intent**，由已知遊戲規則驗證；第二篇才進一步把「玩家實際會怎麼玩」視為隨機環境並建模 human/agent behavior distribution。第二篇主要替換葉節點與 reward 的來源。本篇的關卡庫、semantic verifier 標籤與 value head 亦可作為第二篇世界模型的暖啟動資料。

7 相關工作定位

搜尋式 PCG：Summerville et al. [2]；RL 式 PCG：PCGRL [3]、Nam et al. [4]、EDRL [5]；LLM 關卡生成：MarioGPT [1]；expert iteration：ExIt [6]、AlphaZero [7]；大動作空間規劃：Sampled MuZero [8]、Gumbel MuZero [9]、SBS [10]；LLM + 樹搜尋：TS-LLM [11]。本篇要形成的交集是：**條件式關卡 LLM × spatial action-semantic PATH × executable STRUCTURE-PATH grounding × Gumbel sampled-action 搜尋 × 蒸餾回 LLM**。其中 A* / player-state search 的角色不是發明 PATH semantics，而是把既有 PATH action tokens 落地成可執行的 behavioral constraint 與 terminal ground truth。

參考文獻

- [1] S. Sudhakaran et al., “MarioGPT: Open-Ended Text2Level Generation through Large Language Models,” NeurIPS 2023.
- [2] A. Summerville, S. Philip, M. Mateas, “MCMCTS PCG 4 SMB: Monte Carlo Tree Search to Guide Platformer Level Generation,” AIIDE 2015.
- [3] A. Khalifa, P. Bontrager, S. Earle, J. Togelius, “PCGRL: Procedural Content Generation via Reinforcement Learning,” AIIDE 2020.
- [4] S. Nam, C.-H. Hsueh, P. Rerkjirattikal, K. Ikeda, “Using Reinforcement Learning to Generate Levels of Super Mario Bros. with Quality and Diversity,” *IEEE Trans. Games*, 2024.
- [5] T. Shu, J. Liu, G. N. Yannakakis, “Experience-Driven PCG via Reinforcement Learning: A Super Mario Bros Study,” IEEE CoG 2021.
- [6] T. Anthony, Z. Tian, D. Barber, “Thinking Fast and Slow with Deep Learning and Tree Search,” NeurIPS 2017.
- [7] D. Silver et al., “A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play,” *Science*, 2018.
- [8] T. Hubert et al., “Learning and Planning in Complex Action Spaces,” ICML 2021.
- [9] I. Danihelka, A. Guez, J. Schrittwieser, D. Silver, “Policy Improvement by Planning with Gumbel,” ICLR 2022.
- [10] W. Kool, H. van Hoof, M. Welling, “Stochastic Beams and Where to Find Them,” ICML 2019.
- [11] X. Feng et al., “AlphaZero-like Tree-Search can Guide Large Language Model Decoding and Training,” 2023.
- [12] Y. Niu et al., “LightZero: A Unified Benchmark for Monte Carlo Tree Search in General Sequential Decision Scenarios,” NeurIPS 2023 Datasets and Benchmarks.